

Authentication & Deployment

Backend Development

Session vs. Token Authentication

Session-based auth stores a session identifier server-side (often in a database or in-memory store) and gives the client a cookie referencing it; the server looks up the session on every request to identify the user. Token-based auth (commonly JWT — JSON Web Tokens) instead issues the client a self-contained, signed token that encodes the user's identity directly; the server verifies the token's signature on each request without needing to look anything up in a store. Sessions are simpler to revoke immediately (just delete the session server-side) but require server-side state; tokens scale more easily across multiple servers but are harder to revoke before they naturally expire.

Role-Based Access Control

Most real applications need more than 'logged in or not' — different users need different permissions. Role-based access control (RBAC) assigns each user one or more roles (student, college admin, trainer, company admin, in this app's case), and every protected action checks the current user's role before proceeding, rejecting the request (typically with 403 Forbidden) if their role doesn't permit it. Good RBAC design checks permissions as close to the actual action as possible — not just hiding a button in the UI, since a determined user can call the API directly, bypassing any UI-only restriction.

Environment Configuration for Auth

Authentication secrets — password hashing salts, JWT signing secrets, OAuth client secrets — are exactly the kind of value that must never be hard-coded or committed to version control. These belong in environment variables, injected differently per environment (a throwaway value in development, a strong randomly-generated secret in production, never shared between the two). Password hashing itself should always use an algorithm purpose-built for it (like bcrypt) rather than a general-purpose hash function — general hashes are fast, which is exactly the wrong property for password storage, since it makes brute-force attacks cheaper.

Shipping an Authenticated API

Deploying a backend with real authentication adds a few extra concerns beyond a basic deployment: HTTPS is non-negotiable, since credentials and tokens sent over plain HTTP can be intercepted; cookies (if using session auth) need the correct security flags set (HttpOnly to block JavaScript access, Secure to require HTTPS, SameSite to limit cross-site sending); and rate limiting on login endpoints specifically helps prevent brute-force password guessing. None of these are optional hardening steps for 'later' — they're part of what makes an authenticated

API actually safe to expose to real users.

Learn More

OWASP — Authentication Cheat Sheet

https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html

jwt.io — JSON Web Tokens Introduction

<https://jwt.io/introduction>

MDN Web Docs — Using HTTPS

<https://developer.mozilla.org/en-US/docs/Web/Security>